Game Backend API Documentation

This document provides a complete and up-to-date overview of all API endpoints for the Unreal Engine game ecosystem backend. It includes endpoint paths, HTTP methods, descriptions, authentication requirements, request parameters, request bodies, response formats, and error handling. This is designed for web developers to integrate with frontend (web/mobile) or game clients.

The backend now includes full support for **comments** on posts and **friends** system (send/accept/reject/block requests, list friends, remove friends).

Base URL

- Development: `http://localhost:3000`
- Production: `https://your-domain.com` (configure accordingly)

All endpoints are prefixed with `/api` unless specified otherwise (e.g., health checks).

Authentication

- **JWT Token**: Required for most endpoints. Include in headers as `Authorization: Bearer <token>`.
- **How to Obtain Token**:
  - Via `/api/auth/signup` or `/api/auth/login`.
- **Optional Authentication**: Some endpoints (e.g., public posts, comments) support optional auth for enhanced features like user-specific data (e.g., like status).
- **Dev Mode**: `/api/auth/dev-login` is available only in development for testing (bypasses password).
- **Error Responses for Auth**:
  - 401: Unauthorized (missing/invalid token).
  - 403: Forbidden (insufficient permissions).

General Notes

- **Request/Response Format**: JSON (use `Content-Type: application/json`).
- **Error Handling**: Standardized errors with `success: false`, `message`, `timestamp`, and optional `errors` array.

- **Pagination**: For list endpoints, use `limit` (default: 20/50/100) and `offset` (default: 0). Response includes `pagination` object with `total`, `limit`, `offset`, `has_more`.
- **Rate Limiting**: Applied globally (configurable via env vars).
- **CORS**: Configured for specific origins (env var `CORS_ORIGIN`).
- **Database**: Supabase (PostgreSQL) with Row Level Security (RLS) for security.
- **Health Checks**: Non-API prefixed.
- **Success Responses**: Include `success: true`, `message`, `data`, `timestamp`.

Health Check Endpoints

Public, no authentication required.

GET /health

- **Description**: Check server status.
- **Parameters**: None.
- **Response** (200 OK):

```
{
  "status": "OK",
  "timestamp": "2025-12-21T07:00:00.000Z",
  "service": "Game Backend API"
}
```

- **Errors**: None (always 200 if server is up).

GET /db-status

- **Description**: Check database connection.
- **Parameters**: None.
- **Response** (200 OK):

```
{
  "status": "Database connected successfully"
}
```

- **Errors**:
  - 500: Database connection failed (with details).

Authentication Endpoints (/api/auth)

POST /signup

- **Description**: Register a new user.
- **Authentication**: None.
- **Request Body**:

```
{
  "email": "string (required, unique)",
  "password": "string (required)",
  "username": "string (required, unique)"
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "User created successfully",
  "data": {
    "token": "string (JWT)",
    "userId": "uuid",
    "username": "string",
    "email": "string",
    "level": 1,
    "remnant_count": 100
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing fields or signup failed.
  - 409: Username already taken.
  - 500: Internal error.

POST /login

- **Description**: Login with email/password.
- **Authentication**: None.
- **Request Body**:

```
{
  "email": "string (required)",
  "password": "string (required)"
}
```

- **Response** (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "string (JWT)",
    "userId": "uuid",
    "username": "string",
    "level": "integer",
    "remnant_count": "integer",
    "email": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing fields.
  - 401: Invalid credentials.
  - 404: Profile not found.
  - 500: Internal error.

POST /verify-token

- **Description**: Validate JWT token and get user info.
- **Authentication**: Required.
- **Request Body**: None.
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Token is valid",
  "data": {
    "userId": "uuid",
    "username": "string",
    "level": "integer",
    "remnant_count": "integer",
    "avatar_url": "string (optional)",
    "email": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 401: No token or invalid/expired.
  - 500: Internal error.

POST /dev-login

- **Description**: Development-only login (bypasses password).
- **Authentication**: None.
- **Request Body**:

```
{
  "email": "string (required)"
}
```

- **Response** (200 OK): Same as login.
- **Errors**:
  - 400: Missing email.
  - 403: Not in development mode.
  - 404: User not found.
  - 500: Internal error.

Profile Endpoints (/api/profile)

All require authentication except search.

GET /:userId

- **Description**: Get user profile including purchases.
- **Parameters**: `userId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "userId": "uuid",
    "username": "string",
    "level": "integer",
    "remnant_count": "integer",
    "avatar_url": "string (optional)",
    "bio": "string (optional)",
```

```
      "created_at": "string",
      "last_active": "string",
      "purchased_items": ["array of strings (item_ids)"]
    },
    "timestamp": "string"
  }
```

- **Errors**:
  - 400: Missing userId.
  - 404: Profile not found.
  - 500: Internal error.

PUT /:userId

- **Description**: Update user profile (own only).
- **Parameters**: `userId` (path, uuid, required).
- **Request Body**:

```
{
  "username": "string (optional, unique)",
  "avatar_url": "string (optional)",
  "bio": "string (optional)"
}
```

- **Response** (200 OK):

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "success": true },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Invalid fields or no updates.
  - 403: Not own profile.
  - 409: Username taken.
  - 500: Internal error.

PATCH /:userId/game-stats

- **Description**: Update game stats (level, currency). Supports set/increment/decrement.

- **Parameters**: `userId` (path, uuid, required).
- **Request Body**:

```
{
  "level": "integer (optional)",
  "remnant_count": "integer (optional)",
  "operation": "string (set/increment/decrement, default: set)"
}
```

- **Response** (200 OK):

```
{
  "success": true,
  "message": "Game stats updated successfully",
  "data": {
    "success": true,
    "level": "integer (if updated)",
    "remnant_count": "integer (if updated)"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing userId, no stats, insufficient remnants.
  - 500: Internal error.

GET /:userId/status

- **Description**: Get user's online status (online if active <5 min ago).
- **Parameters**: `userId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Success",
  "data": {
    "userId": "uuid",
    "username": "string",
    "avatar_url": "string (optional)",
    "last_active": "string",
    "is_online": "boolean"
  },
```

```
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing userId.
  - 404: User not found.
  - 500: Internal error.

GET /search/users

- **Description**: Search users by username (public).
- **Parameters** (query):
  - `query` : string (required, min 2 chars).
  - `limit` : integer (default: 20).
  - `offset` : integer (default: 0).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Users retrieved successfully",
  "data": {
    "users": [
      {
        "user_id": "uuid",
        "username": "string",
        "avatar_url": "string",
        "level": "integer",
        "bio": "string"
      }
    ],
    "count": "integer"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Invalid query.
  - 500: Internal error.

Purchases Endpoints (/api/purchases)

All require authentication.

GET /:userId

- **Description**: Get user's purchases (own or admin).
- **Parameters**: `userId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Purchases retrieved successfully",
  "data": [
    {
      "id": "uuid",
      "user_id": "uuid",
      "item_type": "string",
      "item_id": "string",
      "price": "integer",
      "purchased_at": "string",
      "created_at": "string"
    }
  ],
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing userId.
  - 403: Not own purchases (non-admin).
  - 500: Internal error.

POST /

- **Description**: Create a purchase (deducts currency if price >0).
- **Request Body**:

```
{
  "userId": "uuid (required)",
  "item_type": "string (required)",
  "item_id": "string (required)",
  "price": "integer (optional, default 0)"
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "Purchase completed successfully",
  "data": {
    "id": "uuid",
    "user_id": "uuid",
    "item_type": "string",
    "item_id": "string",
    "price": "integer",
    "purchased_at": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing fields or insufficient remnants.
  - 403: Not own userId.
  - 409: Item already purchased.
  - 404: Profile not found.
  - 500: Internal error.

POST /check-ownership/:userId

- **Description**: Check if user owns specific items.
- **Parameters**: `userId` (path, uuid, required).
- **Request Body**:

```
{
  "item_ids": ["array of strings (required)"]
}
```

- **Response** (200 OK):

```
{
  "success": true,
  "message": "Ownership check completed",
  "data": {
    "user_id": "uuid",
    "ownership": { "item_id1": true, "item_id2": false },
    "owned_items": ["array of owned item_ids"]
  },
```

```
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing fields or invalid array.
  - 500: Internal error.

GET /history/:userId

- **Description**: Get purchase history with pagination.
- **Parameters**: `userId` (path, uuid, required).
- **Query Params**:
  - `limit` : integer (default: 20).
  - `offset` : integer (default: 0).
  - `item_type` : string (optional filter).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Purchase history retrieved successfully",
  "data": {
    "purchases": ["array of purchase objects (same as GET /:userId)"]
    "pagination": { "total": "integer", "limit": "integer", "offset":
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing userId.
  - 500: Internal error.

Chat Endpoints (/api/chat)

All require authentication.

POST /send

- **Description**: Send a message (private or channel).
- **Request Body**:

```
{
  "receiver_id": "uuid (required for private)",
  "message": "string (required, max 1000 chars)",
  "channel": "string (default: private)"
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "Message sent successfully",
  "data": {
    "chat_id": "uuid",
    "sender_id": "uuid",
    "receiver_id": "uuid (optional)",
    "message": "string",
    "channel": "string",
    "created_at": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Invalid message or too long.
  - 404: Receiver not found (private).
  - 500: Internal error.

GET /

- **Description**: Get messages (own only) with filters.
- **Query Params**:
  - `withUserId` : uuid (filter private convo).
  - `channel` : string (filter channel).
  - `limit` : integer (default: 50).
  - `offset` : integer (default: 0).
  - `startDate` : string (ISO, optional).
  - `endDate` : string (ISO, optional).
- **Response** (200 OK):

```
{
  "success": true,
```

```
  "message": "Messages retrieved successfully",
  "data": {
    "messages": [
      {
        "id": "uuid",
        "sender_id": "uuid",
        "sender_username": "string",
        "sender_avatar": "string",
        "receiver_id": "uuid",
        "receiver_username": "string",
        "receiver_avatar": "string",
        "message": "string",
        "channel": "string",
        "created_at": "string"
      }
    ],
    "pagination": { ... }
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Fetch failed.
  - 500: Internal error.

GET /conversations

- **Description**: Get list of private conversations.
- **Parameters**: None.
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Conversations retrieved successfully",
  "data": [
    {
      "partner_id": "uuid",
      "partner_username": "string",
      "partner_avatar": "string",
      "last_message": "string",
      "last_message_at": "string",
      "unread_count": "integer (0 by default)"
    }
```

```
  ],
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Fetch failed.
  - 500: Internal error.

GET /channel/:channel

- **Description**: Get channel messages.
- **Parameters**: `channel` (path, string, required).
- **Query Params**:
  - `limit` : integer (default: 100).
  - `offset` : integer (default: 0).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Channel messages retrieved successfully",
  "data": [
    {
      "id": "uuid",
      "sender_id": "uuid",
      "sender_username": "string",
      "sender_avatar": "string",
      "message": "string",
      "channel": "string",
      "created_at": "string"
    }
  ],
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing channel or fetch failed.
  - 500: Internal error.

Posts Endpoints (/api/posts)

GET /

- **Description**: Get posts with pagination (public, optional auth for like status).
- **Authentication**: Optional.
- **Query Params**:
  - `limit` : integer (default: 20).
  - `offset` : integer (default: 0).
  - `author_id` : uuid (filter by author).
  - `sort_by` : string (newest/popular, default: newest).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Posts retrieved successfully",
  "data": {
    "posts": [
      {
        "id": "uuid",
        "author_id": "uuid",
        "author_username": "string",
        "author_avatar": "string",
        "author_level": "integer",
        "content": "string",
        "attachments": ["array of strings"],
        "likes": "integer",
        "comments_count": "integer",
        "user_liked": "boolean (if authenticated)",
        "created_at": "string"
      }
    ],
    "pagination": { ... }
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Fetch failed.
  - 500: Internal error.

POST /

- **Description**: Create a post.
- **Authentication**: Required.
- **Request Body**:

```
{
  "content": "string (required, max 5000 chars)",
  "attachments": ["array of strings (optional)"]
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "Post created successfully",
  "data": {
    "id": "uuid",
    "author_id": "uuid",
    "author_username": "string",
    "author_avatar": "string",
    "content": "string",
    "attachments": ["array"],
    "likes": 0,
    "created_at": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Invalid content.
  - 500: Internal error.

POST /:postId/like

- **Description**: Like/unlike a post (toggles).
- **Authentication**: Required.
- **Parameters**: `postId` (path, uuid, required).
- **Request Body**: None.
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Post liked/unliked successfully",
  "data": {
    "liked": "boolean",
    "likes": "integer"
  },
```

```
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing postId.
  - 404: Post not found.
  - 500: Internal error.

GET /:postId

- **Description**: Get single post (public).
- **Authentication**: Optional.
- **Parameters**: `postId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Post retrieved successfully",
  "data": {
    "id": "uuid",
    "author_id": "uuid",
    "author_username": "string",
    "author_avatar": "string",
    "author_level": "integer",
    "content": "string",
    "attachments": ["array"],
    "likes": "integer",
    "comments_count": "integer",
    "total_likes": "integer",
    "created_at": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing postId.
  - 404: Post not found.
  - 500: Internal error.

DELETE /:postId

- **Description**: Delete a post (own only).

- **Authentication**: Required.
- **Parameters**: `postId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Post deleted successfully",
  "data": { "success": true },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing postId.
  - 403: Not own post.
  - 404: Post not found.
  - 500: Internal error.

Comments Endpoints (/api/comments)

POST /

- **Description**: Create a comment on a post (own only).
- **Authentication**: Required.
- **Request Body**:

```
{
  "postId": "uuid (required)",
  "content": "string (required, max 2000 chars)"
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "Comment created successfully",
  "data": {
    "id": "uuid",
    "post_id": "uuid",
    "user_id": "uuid",
    "author_username": "string",
    "author_avatar": "string (optional)",
    "content": "string",
```

```
    "created_at": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing fields or invalid content.
  - 404: Post not found.
  - 500: Internal error.

GET /post/:postId

- **Description**: Get comments for a post (public).
- **Parameters**: `postId` (path, uuid, required).
- **Query Params**:
  - `limit` : integer (default: 20).
  - `offset` : integer (default: 0).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Comments retrieved successfully",
  "data": {
    "comments": [
      {
        "id": "uuid",
        "post_id": "uuid",
        "user_id": "uuid",
        "author_username": "string",
        "author_avatar": "string (optional)",
        "content": "string",
        "created_at": "string"
      }
    ],
    "pagination": { "total": "integer", "limit": "integer", "offset":
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing postId.
  - 500: Internal error.

DELETE /:commentId

- **Description**: Delete a comment (own only).
- **Authentication**: Required.
- **Parameters**: `commentId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Comment deleted successfully",
  "data": { "success": true },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing commentId.
  - 403: Not own comment.
  - 404: Comment not found.
  - 500: Internal error.

Friends Endpoints (/api/friends)

All require authentication.

POST /request

- **Description**: Send a friend request.
- **Request Body**:

```
{
  "friendId": "uuid (required, not self)"
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "Friend request sent",
  "data": {
    "id": "uuid",
    "user_id": "uuid",
```

```
    "friend_id": "uuid",
    "status": "pending",
    "created_at": "string"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Invalid friendId.
  - 404: User not found.
  - 409: Request already exists.
  - 500: Internal error.

POST /respond

- **Description**: Respond to a friend request (accept/reject/block).
- **Request Body**:

```
{
  "requestId": "uuid (required)",
  "action": "string (accept/reject/block, required)"
}
```

- **Response** (200 OK):

```
{
  "success": true,
  "message": "Request processed",
  "data": {
    "success": true,
    "status": "accepted/rejected/blocked"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Invalid fields.
  - 403: Not recipient.
  - 404: Request not found.
  - 500: Internal error.

GET /

- **Description**: Get accepted friends list.
- **Query Params**:
  - `limit` : integer (default: 20).
  - `offset` : integer (default: 0).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Friends retrieved successfully",
  "data": {
    "friends": [
      {
        "friend_id": "uuid",
        "username": "string",
        "avatar_url": "string (optional)",
        "level": "integer",
        "status": "accepted"
      }
    ],
    "pagination": { ... }
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 500: Internal error.

GET /pending

- **Description**: Get pending incoming friend requests.
- **Parameters**: None.
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Pending requests retrieved",
  "data": [
    {
      "request_id": "uuid",
      "sender_id": "uuid",
      "sender_username": "string",
      "sender_avatar": "string (optional)",
      "status": "pending"
```

```
    }
  ],
  "timestamp": "string"
}
```

- **Errors**:
  - 500: Internal error.

DELETE /:friendId

- **Description**: Remove a friend or cancel request.
- **Parameters**: `friendId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Friend removed successfully",
  "data": { "success": true },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing friendId.
  - 500: Internal error.

Events Endpoints (/api/events)

All public except create/join.

GET /

- **Description**: Get events with status filter.
- **Query Params**:
  - `status` : string (upcoming/ongoing/past, default: upcoming).
  - `limit` : integer (default: 20).
  - `offset` : integer (default: 0).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Events retrieved successfully",
```

```
  "data": {
    "events": [
      {
        "id": "uuid",
        "title": "string",
        "description": "string",
        "image_url": "string (optional)",
        "start_time": "string",
        "end_time": "string",
        "rewards": ["array"],
        "created_at": "string",
        "status": "string (upcoming/ongoing/past)"
      }
    ],
    "pagination": { ... }
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Fetch failed.
  - 500: Internal error.

GET /active

- **Description**: Get ongoing events.
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Active events retrieved successfully",
  "data": [
    {
      "id": "uuid",
      "title": "string",
      "description": "string",
      "start_time": "string",
      "end_time": "string",
      "image_url": "string",
      "rewards": ["array"]
    }
  ],
```

```
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Fetch failed.
  - 500: Internal error.

GET /:eventId

- **Description**: Get single event.
- **Parameters**: `eventId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Event retrieved successfully",
  "data": {
    "id": "uuid",
    "title": "string",
    "description": "string",
    "image_url": "string",
    "start_time": "string",
    "end_time": "string",
    "rewards": ["array"],
    "created_at": "string",
    "status": "string",
    "participants_count": "integer (0 by default)"
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing eventId.
  - 404: Event not found.
  - 500: Internal error.

POST /

- **Description**: Create event (admin only; role check commented out for testing).
- **Authentication**: Required.
- **Request Body**:

```
{
  "title": "string (required)",
  "description": "string (required)",
  "start_time": "string (ISO, required)",
  "end_time": "string (ISO, required)",
  "image_url": "string (optional)",
  "rewards": ["array (optional)"]
}
```

- **Response** (201 Created):

```
{
  "success": true,
  "message": "Event created successfully",
  "data": {
    "id": "uuid",
    "title": "string",
    "description": "string",
    "start_time": "string",
    "end_time": "string",
    "image_url": "string",
    "rewards": ["array"]
  },
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Missing fields or invalid dates.
  - 500: Internal error.

POST /:eventId/join

- **Description**: Join an ongoing event.
- **Authentication**: Required.
- **Parameters**: `eventId` (path, uuid, required).
- **Response** (200 OK):

```
{
  "success": true,
  "message": "Joined event successfully",
  "data": { "success": true },
```

```
  "timestamp": "string"
}
```

- **Errors**:
  - 400: Not started/ended.
  - 404: Event not found.
  - 409: Already joined.
  - 500: Internal error.

Additional Integration Notes

- **Unreal Engine**: Use HTTP requests with JWT in headers. Parse JSON responses into structs (e.g., FPlayerInfo).
- **Error Codes**: Standard HTTP codes with JSON details.
- **Security**: Use HTTPS in production. Validate inputs client-side.
- **Testing**: Use Postman or `test-api.js` script.
- **Deployment**: Configure env vars for Supabase, JWT, CORS, etc.
- **New Features**:
  - Comments: Fully integrated with posts (create/get/delete).
  - Friends: Send/respond requests, list friends, remove friends.

This is the complete, current documentation as of December 21, 2025. If you need Swagger/OpenAPI spec or more details, let me know!